UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE AUTOMAÇÃO E SISTEMAS

Daniel Floriani da Silva

Relatório de Estágio

Extração, Transformação e Carregamento de dados: Um projeto prático utilizando a Arquitetura Medallion

Florianópolis

2026

Daniel Floriani da Silva

Extração, Transformação e Carregamento de dados: Um projeto prático utilizando a Arquitetura Medallion

Relatório submetido à Universidade Federal de Santa Catarina como requisito para a aprovação na disciplina **DAS 5501 - Estágio em Controle e Automação Industrial** do curso de Graduação em Engenharia de Controle e Automação.
Orientador: Leandro Buss Becker

Florianópolis

2026

Daniel Floriani da Silva

Extração, Transformação e Carregamento de dados: Um projeto prático utilizando a Arquitetura Medallion

Este relatório de estágio foi julgado no contexto da disciplina DAS5501: Estágio em Controle e Automação Industrial e **APROVADO** na sua forma final pelo Curso de Engenharia de Controle e Automação.

Florianópolis, 15 de abril de 2026

Leandro Buss Becker
Orientador
Universidade Federal de Santa Catarina

Vinicius Rios Fuck
Supervisor na Empresa
EMPRESA

Resumo

Apresenta as informações principais do documento (Descrição geral da empresa (natureza, mercado, processos, etc.), problema-foco atacado no PFC, o que foi feito, principais resultados atingidos, etc.). Deve conter entre 100 e 500 palavras (NBR 6028/2003) em um único parágrafo. Se o documento for escrito em outra língua que não o Português, então é necessário fazer um Resumo **Estendido** em Português, ao invés deste resumo enxuto.

Palavras-chave: No mínimo 3 (três) e separadas por ponto (.)

Abstract

Resumo em língua inglesa. Mesma formatação do Resumo.

Key-words: No mínimo 3 (três) e separadas por ponto (.)

Lista de ilustrações

Figura 1 – Diagrama de Arquitetura do *Pipeline Medallion* 17
Figura 2 – Fluxo de Processamento de Dados entre as Camadas 19
Figura 3 – Trecho de Código - Implementação de Contrato de Dados com Pandera 22
Figura 4 – Trecho de Código - Rotina Lógica de Validação e Reconciliação 24

OPCIONAL

Sumário

1 INTRODUÇÃO 7

Introdução 7
1.1 Objetivos 7
1.2 Metodologia 8
1.3 Estrutura do Relatório 8

2 A EMPRESA E O PROBLEMA 10
2.1 O Problema de Extração e Validação 10

3 ASPECTOS CONCEITUAIS 12
3.1 Pipelines e Qualidade de Dados 12
3.2 Arquitetura *Medallion* e Fragmentação de Ingestão 12
3.3 Camada de Apresentação (*Views*) e Versatilidade 13
3.4 Contratos de Dados e *Schemas* 14
3.5 Estratégias de Validação e Reconciliação de Dados 14

4 A ARQUITETURA DA SOLUÇÃO 16
4.1 Ambiente de Execução e Orquestração 16
4.2 Fluxo e Evolução Estrutural dos Arquivos 16

5 A IMPLEMENTAÇÃO DO *PIPELINE* 20
5.1 Ambiente de Desenvolvimento e Tecnologias 20
5.2 Orquestração e Ingestão (Camada *Bronze*) 20
5.3 Validação e Padronização (Camada *Silver*) 21
5.4 Transformação Analítica e Apresentação (Camadas *Gold* e *View*) . 21
5.5 Monitoramento e Rastreabilidade 22

6 CAPÍTULO 6 25

7 CAPÍTULO 7 26

REFERÊNCIAS 27

APÊNDICE A – GLOSSÁRIO DE TERMOS TÉCNICOS 28

1 Introdução

O presente relatório de estágio aborda o desafio da estruturação e da confiabilidade no consumo de dados corporativos. O problema central a ser resolvido consiste na fragilidade, morosidade e na falta de padronização durante a ingestão manual de dados provenientes de fontes externas críticas para o negócio, specificamente os relatórios em PDF (*Portable Document Format*) de duas fontes confidenciais, tratadas aqui como Fonte X e Fonte Y.

Este trabalho foi desenvolvido durante o Estágio Obrigatório realizado na empresa **Veeries**, no período de **outubro de 2025** a **abril de 2026**. O projeto insere-se no contexto do curso de Engenharia de Controle e Automação da Universidade Federal de Santa Catarina (UFSC), aplicando conhecimentos da grande área de informática industrial, especificamente no que tange a estruturas, processamento e automação de fluxos de dados, para a resolução de problemas práticos de mercado.

A confiabilidade dos dados é um fator crítico para a tomada de decisão nas organizações. A ausência de um tratamento automatizado e adequado pode gerar inconsistências silenciosas que impactam negativamente as análises gerenciais e estratégicas da equipe de mercado. A escolha deste tema justifica-se pela oportunidade de aplicar boas práticas de engenharia de *software* e de dados para mitigar esses riscos de intervenção humana, garantindo a integridade e a qualidade das informações disponibilizadas. Cumpre destacar que, ao longo de todo o projeto, atuei de forma direta e integral na idealização da arquitetura, na modelagem lógica e na codificação dos *scripts* que compõem a solução final.

1.1 Objetivos

O objetivo geral deste estágio foi desenvolver e implementar um *pipeline* (uma esteira automatizada de fluxo de dados) para a extração, processamento e validação das informações provenientes das fontes X e Y.

Como objetivos específicos, destacam-se:

- Projetar o fluxo de dados fundamentado na Arquitetura *Medallion* (um modelo de organização em camadas progressivas de qualidade: *Bronze, Silver* e *Gold*).
- Desenvolver rotinas programáticas de extração bruta de dados (*parsers*) a partir de arquivos de formato complexo e não estruturado.
- Implementar rigorosas validações de qualidade e limpeza estrutural utilizando *schemas* (contratos técnicos que definem o formato esperado da informação).

A argumentação principal deste trabalho é que a adoção da Arquitetura *Medallion*, aliada à implementação de contratos de dados estritos na transição entre as camadas de processamento, é capaz de garantir um fluxo de informação resiliente, rastreável e auditável, solucionando definitivamente o gargalo operacional da despadronização inicial.

1.2 Metodologia

A metodologia adotada possui um caráter eminentemente prático e aplicado, com foco no relato das técnicas e ferramentas utilizadas no desenvolvimento da automação, adequando-se ao rigor teórico esperado para um relatório de estágio técnico. Trata-se de um projeto de arquitetura de *software* em nível de produção.

Para a sua consecução, foram arquitetados e codificados *scripts* em linguagem Python atuando como orquestradores (`main_source_X.py` e `main_source_Y.py`). Estes orquestram módulos de processamento progressivos para transitar a informação entre as camadas do sistema (`l0_process.py`, `l1_process.py`, `l2_process.py` e `view_monthly_process.py`), assegurados por regras de validação tipada e reconciliação matemática.

Como alternativa de solução frente aos modelos tradicionais de integração de dados executados em lote único (onde extração, transformação e carregamento ocorrem sem zonas de quarentena ou auditoria intermediária), optou-se pela Arquitetura *Medallion*. Essa abordagem permite o armazenamento dos arquivos em estados progressivos de refinamento em nuvem, facilitando a auditoria contínua e a manutenção paliativa pontual sem a necessidade de reprocessar todo o sistema em caso de falhas na fonte (GUJJALA, 2023).

1.3 Estrutura do Relatório

Para descrever o desenvolvimento e os resultados alcançados, o plano adotado para este relatório está organizado da seguinte maneira:

- O **Capítulo 2** descreve o contexto da empresa, detalhando a divisão das equipes, os conceitos primários sobre dados e os pormenores do problema motivador do estágio.
- O **Capítulo 3** apresenta, de forma concisa e prática, o embasamento bibliográfico da Arquitetura *Medallion* e das técnicas de validação de dados utilizadas na solução.
- O **Capítulo 4** descreve a arquitetura macro da solução, o ambiente de execução e a evolução estrutural (granularidade) dos arquivos ao longo do fluxo.
- O **Capítulo 5** mergulha na implementação técnica, expondo os métodos de codificação, o *parsing* de arquivos e a transição algorítmica entre as camadas do projeto.

- O **Capítulo 6** expõe os resultados e promove uma análise crítica profunda sobre os ganhos reais de confiabilidade e desempenho alcançados na organização.
- Por fim, o **Capítulo 7** reúne a síntese interpretada dos resultados, avalia o cumprimento dos objetivos traçados e aponta perspectivas para trabalhos futuros.

2 A Empresa e o Problema

A Veeries é uma empresa inserida no agronegócio. Oferece produtos relacionados à análise da produção, fluxo e operações de commodities, bem como fluxo e operações dos suprimentos e infra-estrutura relacionados a este mercado. A empresa conta com cerca de 20 funcionários e pode ser subdividida em duas equipes: uma de Analistas de Mercado e outra de Desenvolvedores, como assim são chamados na organização.

Próximo ao produto final, os analistas são responsáveis por analisar as informações coletadas, fazer projeções e montar relatórios. Estritamente ligados ao processamento da informação, o time de desenvolvedores é composto por engenheiros, cientistas da computação e estudantes de áreas correlatas. São responsáveis pela melhoria do processo, desde a obtenção dos dados até a apresentação ao cliente.

ETL, sigla para *Extraction, Transform, Load* (extração, transformação e carregamento), é o processo de integração de dados provindo de uma ou mais fontes de informação. Inclui processos de limpeza, organização, validação e carregamento em um repositório organizado e é útil para consolidar e garantir dados brutos.

Os dados da fonte de informação, sobretudo o seu nível de organização, ditam o quão complexa vai ser sua extração. O grau de dificuldade aumenta conforme diminui a clareza da estrutura da informação. Comumente é utilizada a classificação de dados estruturados, dados semi-estruturados e dados não estruturados. Dados estruturados (bancos de dados relacionais, planilhas Excel, por exemplo) costumam trazer pouco esforço de processamento por conter sua estrutura quase que pronta para consumo em ferramentas de análise. Dados semi-estruturados não se encaixam em tabelas, mas possuem marcadores e metadados (tais como saída de leitura de sensores, JSON, XML, logs de sistemas, e-mails). Já os dados não estruturados não possuem formato definido (documentos de texto livre, PDFs, imagens, vídeos) e por isso exigem mais esforço no processamento.

Este projeto de ETL lida com um tipo de dado que se classificaria entre dados não estruturados e semi-estruturados, pois a fonte de informação primária está disposta em tabelas documentadas em arquivos de texto PDF, onde é possível localizar a posição relativa dos elementos de texto.

2.1 O Problema de Extração e Validação

Historicamente na Veeries, a extração de dados oriundos de duas fontes externas essenciais para o mercado (tratadas de forma confidencial neste relatório como Fonte X e Fonte Y) representava um severo gargalo operacional. Os arquivos, recebidos com uma

frequência mensal, exigiam uma atividade de extração estritamente manual por parte da equipe de analistas.

O fluxo de trabalho consistia na consulta visual dos arquivos em formato PDF e na transcrição manual dos valores e tabelas para planilhas do Excel. Por se tratar de um processo rudimentar e não estruturado, a atividade era extremamente morosa: consumia-se, em média, uma hora de trabalho humano focado apenas para a extração de cada relatório que chegava, sem contabilizar o tempo adicional necessário para higienizar e consumir os dados dessas planilhas despadronizadas.

Além do elevado custo de tempo, esse modelo manual era altamente suscetível a falhas humanas. Não havia uma validação sistêmica clara ou **contratos de dados (regras estritas de formatação)** que garantissem que a extração manual havia ocorrido sem erros de digitação ou deslocamento de linhas. Como consequência direta do esforço desproporcional exigido e da baixa confiabilidade do processo, essas fontes de informação acabavam sendo pouco consideradas ou subutilizadas nas análises de mercado da empresa.

É exatamente para solucionar esse gargalo, automatizando a extração mensal dos PDFs e garantindo a confiabilidade da informação por meio de validações estruturais, que o desenvolvimento de um *pipeline* de dados robusto se fez necessário, tema que será detalhado nos capítulos a seguir.

3 Aspectos Conceituais

Para solucionar o problema da extração manual e garantir a confiabilidade dos dados ingeridos pela Veeries, fez-se necessário o uso de padrões arquiteturais consolidados na área de engenharia de dados. Este capítulo apresenta, de forma prática, os conceitos de *pipelines* de dados, a Arquitetura *Medallion* e a utilização de *schemas* como contratos de dados, fundamentando as escolhas técnicas implementadas na solução.

3.1 Pipelines e Qualidade de Dados

Um *pipeline* de dados consiste em um conjunto automatizado de processos que realiza a extração, o processamento, a validação e o carregamento de informações de um sistema de origem para um ambiente de destino (POLISETTY; PENDYALA; PRABHAKARAN, 2023). Em oposição aos processos manuais e rudimentares, a adoção de fluxos automatizados garante a continuidade da informação e mitiga drasticamente a ocorrência de falhas humanas (REIS; HOUSLEY, 2022).

A qualidade dos dados (*Data Quality*) é o objetivo central dessas rotinas estruturadas, sendo uma medida avaliada com base em fatores como precisão, completude, consistência e confiabilidade (POLISETTY; PENDYALA; PRABHAKARAN, 2023). No contexto do processamento de dados não estruturados e semi-estruturados — como a leitura e extração de tabelas em arquivos PDF —, a automação do fluxo é o primeiro passo obrigatório para estabelecer a confiabilidade do que será entregue à equipe de analistas.

3.2 Arquitetura *Medallion* e Fragmentação de Ingestão

Para organizar o *pipeline* de maneira escalável, de fácil manutenção e rastreável, optou-se pela utilização da Arquitetura *Medallion*. Trata-se de um padrão de projeto de organização de dados que divide os arquivos e processamentos em camadas lógicas, com o objetivo de melhorar progressivamente a estrutura e a qualidade das informações a cada nova etapa (Databricks, 2023; GUJJALA, 2023).

Nesta arquitetura, a transição e o refinamento dos dados ocorrem na seguinte divisão:

- **Camada *Bronze* (L0):** Consiste na camada inicial de armazenamento após a extração dos dados originais. Neste projeto, os arquivos provenientes da camada anterior (arquivos PDF) são processados para um formato tabular e, para lidar com

a imprevisibilidade dessas fontes, a camada *Bronze* foi estrategicamente fragmentada em duas subzonas de mesma estrutura: a **L0 Original** e a **L0 Revisada**.

A **L0 Original** recebe o processamento inicial e mantém os arquivos totalmente intactos. Não há nenhuma intervenção nesta subcamada; ela atua como um registro imutável do sistema, garantindo a rastreabilidade e a possibilidade de auditoria exata do que foi extraído (GORELIK, 2019).

A **L0 Revisada** Atua como uma zona de ajuste dos dados. Caso os dados extraídos não passem nas etapas de validação das camadas subsequentes (principalmente na L1) porque a fonte não seguiu o padrão esperado, o dado é reestruturado aqui. Desta maneira, o pipeline pode ser executado novamente sem a necessidade de reprocessar o dado na camada Bronze.

- **Camada *Silver* (L1):** Trata-se da camada intermediária. Nela, os dados tabulares provenientes da L0 Revisada são submetidos a rigorosos processos de limpeza, padronização, deduplicação e, principalmente, validação contra contratos predefinidos. É o estágio onde a informação deixa de ser um dado bruto e passa a ser um dado confiável e estruturado (GUJJALA, 2023).

- **Camada *Gold* (L2):** É a camada final de processamento do repositório. Os dados, já limpos e validados, são agregados e transformados conforme as regras abrangentes da organização, ficando otimizados e prontos para o consumo analítico (Databricks, 2023).

3.3 Camada de Apresentação (*Views*) e Versatilidade

Embora a camada *Gold* finalize o tratamento da informação, a entrega direta de tabelas físicas aos usuários analistas pode gerar engessamento. Para solucionar isso, as modernas arquiteturas de engenharia de dados adotam uma Camada de Apresentação (frequentemente materializada através de *Views* lógicas) operando sobre a camada *Gold* (REIS; HOUSLEY, 2022).

Uma diretriz de projeto fundamental adotada neste trabalho foi a de propagar o máximo de campos e informações viáveis extraídas da fonte primária até a camada anterior à apresentação (L2). A premissa central de preservar a abrangência e a granularidade atômica dos dados garante uma extrema versatilidade ao *pipeline* (KIMBALL; ROSS, 2013).

Se o cliente final (analista de mercado) alterar seus requisitos de negócio e solicitar a visualização de uma coluna extraída do PDF que não havia sido prevista nos relatórios iniciais, essa arquitetura evita retrabalho. Como o dado já foi extraído da fonte, validado na L1 e preservado até a camada L2, não há necessidade de reconstruir os *scripts* de ingestão:

basta editar os limites da *View* para passar a expor a nova métrica. Esse desacoplamento entre armazenamento estruturado e apresentação final eleva a resiliência operacional da solução.

3.4 Contratos de Dados e *Schemas*

Um dos principais desafios na ingestão de arquivos externos é a mudança inesperada na estrutura original dos dados, como o reposicionamento de valores em um PDF. Para evitar que essas alterações corrompam silenciosamente os relatórios finais da empresa, utilizam-se os Contratos de Dados (*Data Contracts*), que são acordos técnicos estritos definindo o formato, a estrutura e a tipagem esperada das informações na transição entre as camadas do *pipeline* (KLEPPMANN, 2017; ARMBRUST et al., 2020).

A principal ferramenta lógica para impor o cumprimento desses contratos é o *Schema* (Esquema). O *schema* especifica claramente os nomes das colunas, os tipos de dados exigidos e as restrições de obrigatoriedade (LIAO; ZHANG; LIN, 2020).

Ao se aplicar a imposição de esquemas (*schema enforcement*) na transição entre a L0 Revisada e a camada *Silver*, o sistema atua como uma barreira de segurança: registros que não cumprem os requisitos estruturais preestabelecidos são bloqueados ou sinalizados para correção. Essa prática é essencial para lidar com a evolução estrutural dos dados, protegendo o sistema contra quebras e garantindo que apenas informações íntegras fluam para as *Views* de apresentação (ARMBRUST et al., 2020).

3.5 Estratégias de Validação e Reconciliação de Dados

Para assegurar a integridade da informação propagada pelo *pipeline*, não basta apenas impor a tipagem de dados através de *schemas*; é fundamental garantir a coerência matemática e lógica dos valores extraídos da Fonte X e da Fonte Y. Para isso, são empregadas técnicas de reconciliação de dados, divididas conceitualmente em validações horizontais, verticais e verificações de duplicidade (POLISETTY; PENDYALA; PRABHAKARAN, 2023).

Na transição dos dados brutos (L0) para a camada intermediária estruturada (L1), aplicam-se duas validações matemáticas principais:

- **Validação Vertical:** Atua no nível das colunas. O seu objetivo é garantir que a soma de todos os valores de uma determinada métrica extraída corresponda exatamente ao valor totalizador geral divulgado pela própria fonte em uma linha de totalização de todas colunas de interesse.

- **Validação Horizontal:** Opera no nível da instância (linha). Esta verificação assegura que, para um dado registro específico, a soma das variáveis parciais que o compõem resulte no valor idêntico à coluna de total correspondente àquela mesma linha.

Como os dados primários são originados de processos de *parsing* de arquivos PDF, anomalias de formatação ou problemas de arredondamento da própria fonte emissora podem ocorrer. Por esta razão, tanto a validação vertical quanto a horizontal são projetadas com margens de tolerâncias matemáticas específicas e ajustáveis. Isso garante que o *pipeline* seja resiliente a flutuações decimais irrelevantes, mantendo a continuidade do processamento sem abrir mão da confiabilidade estrutural.

Por fim, a **validação de unicidade** (*uniqueness validation*) é uma etapa crucial para garantir que não existam registros estritamente duplicados trafegando pelo *pipeline*. Diferente de lógicas de filtragem temporal ou de versão de documentos, a validação de unicidade assegura que uma mesma instância — definida por um conjunto específico de chaves compostas (*composite keys*) — apareça apenas uma vez no modelo.

Essa verificação atua como uma malha fina de segurança em etapas onde ocorrem operações estruturais complexas na modelagem, como o despivoteamento de tabelas para o formato longo (*Melt*) ou cruzamentos entre bases (*Joins*). Tais operações carregam o risco inerente de multiplicar linhas indevidamente em caso de falha algorítmica (fenômeno conhecido como *fan-out*). Validar a unicidade de forma programática protege a integridade atômica do modelo, garantindo que as métricas não sejam contabilizadas em duplicidade nas análises finais (REIS; HOUSLEY, 2022).

4 A Arquitetura da Solução

Para sanar o gargalo operacional imposto pela extração manual de dados não estruturados, foi concebida uma arquitetura baseada em eventos lógicos e armazenamento em nuvem. Este capítulo descreve conceitualmente o novo fluxo de informações da Veeries, detalhando o ambiente de execução, o ciclo de vida dos arquivos e a transição da **granularidade (nível de detalhamento)** dos dados através das camadas do *pipeline*.

4.1 Ambiente de Execução e Orquestração

A infraestrutura de armazenamento do projeto foi estabelecida sobre o ecossistema de nuvem sincronizada da organização (utilizando as plataformas *OneDrive/SharePoint*). Essa escolha garante alta disponibilidade e espelhamento nativo dos arquivos do *pipeline* com as estações de trabalho da equipe.

Como as fontes externas não possuem uma data de publicação exata ou previsível para a emissão dos seus relatórios mensais em PDF, a adoção de um agendador de tarefas tradicional (baseado em tempo cronometrado) resultaria em execuções ociosas ou falhas por ausência de arquivos. Portanto, o acionamento do *pipeline* baseia-se em uma arquitetura orientada a eventos, com orquestração sob demanda.

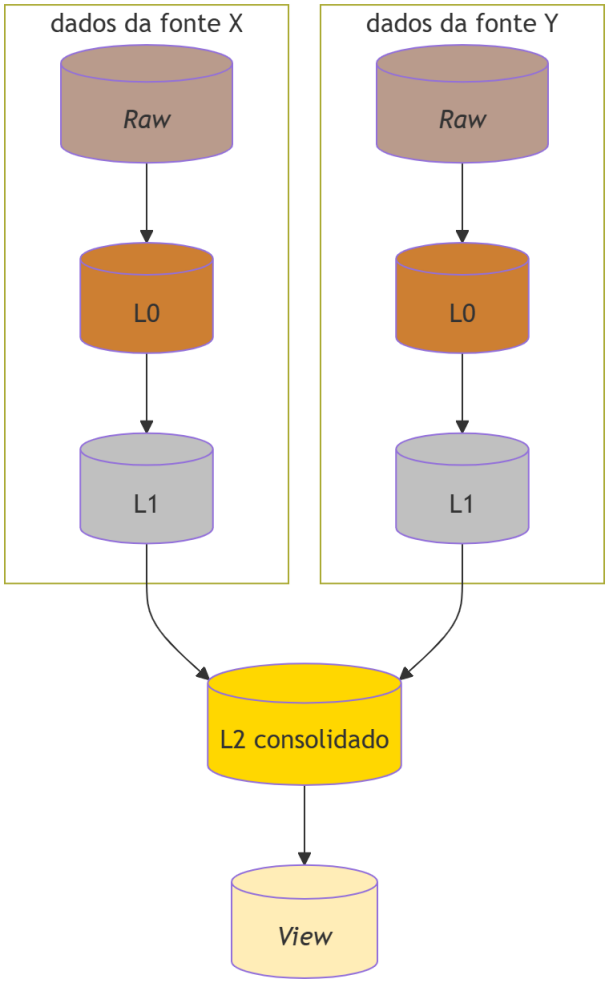
O fluxo se inicia quando o analista responsável pelo monitoramento recebe o documento da fonte (via e-mail ou portais do mercado). O arquivo é então salvo de maneira padronizada no diretório de entrada inicial do *pipeline* (denominado `01_RAW`). Após o armazenamento, o usuário aciona a execução do processamento através de um *bot* no aplicativo de mensagens *Telegram*, o qual atua como o gatilho externo. Este *bot* envia um comando **(via *webhook*, um mecanismo de comunicação em tempo real, ou requisição de API - interface de comunicação entre sistemas)** que inicia a rotina de execução do repositório no *GitHub*, transferindo o processamento pesado para o ambiente em nuvem da aplicação.

A Figura 1 ilustra o diagrama esquemático deste fluxo lógico, desde a captura dos dados crus até a sua disponibilização para os usuários analistas de mercado.

4.2 Fluxo e Evolução Estrutural dos Arquivos

Uma diretriz central desta arquitetura foi o isolamento contínuo das fontes de dados durante as etapas de limpeza e validação, a fim de evitar que anomalias em um relatório comprometessem a ingestão do outro. A Figura 2 detalha visualmente as etapas

Figura 1 – Diagrama de Arquitetura do *Pipeline Medallion*



Fonte: Elaborado pelo autor.

de processamento e a evolução estrutural dos arquivos dentro do *pipeline*.

A transição e consolidação da informação ocorrem de maneira gradativa através das camadas:

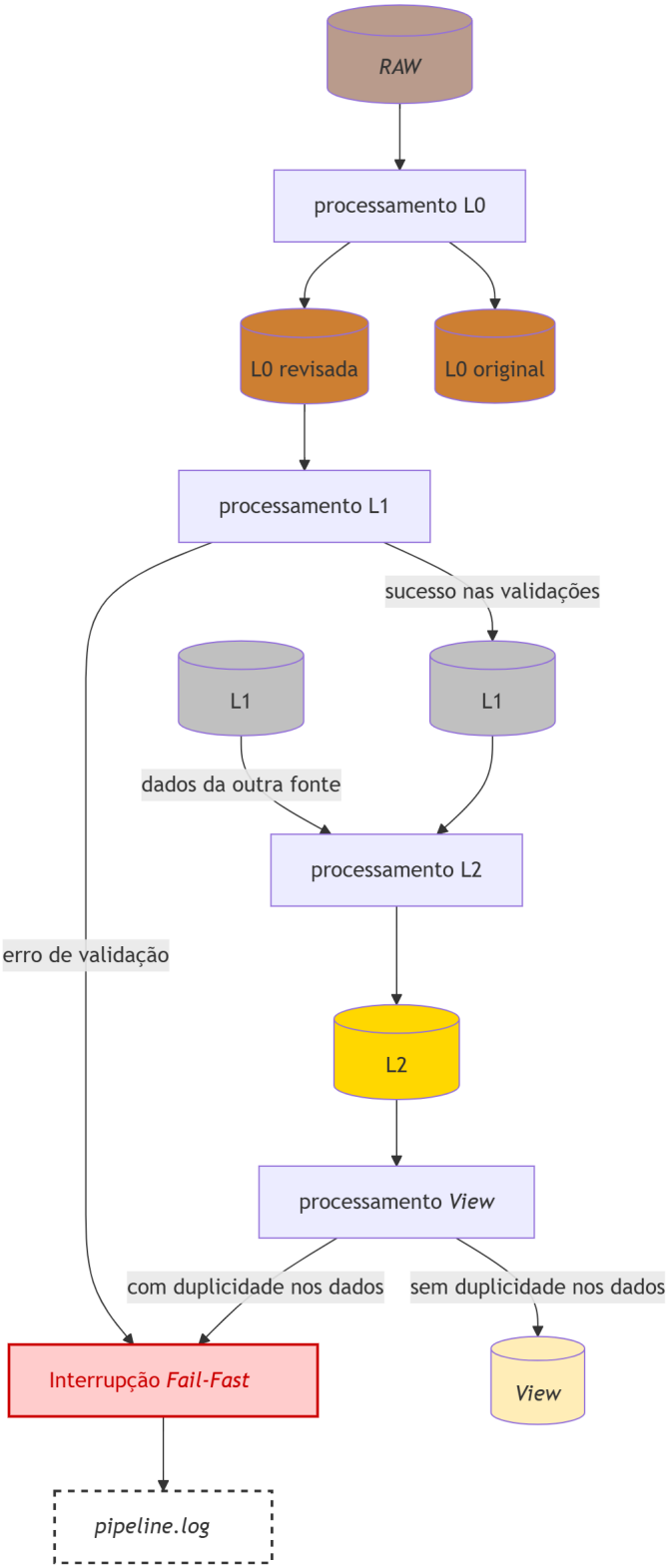
- **Camada *Raw* (Crua):** Os arquivos que ingressam nesta camada preservam seu formato nativo (PDF). A granularidade é estritamente unitária: cada arquivo corresponde a um documento em PDF de uma única fonte de informação, retratando uma única data de emissão e um mês de referência específico. Os arquivos são mantidos em diretórios separados por fonte.
- **Camada *Bronze* (L0 Original e Revisada):** Após o acionamento do ***parser* (algoritmo de extração de texto)**, o dado é convertido do formato PDF para arquivos tabulares no formato .CSV (*Comma-Separated Values*), preservando a granularidade original. Assim, na camada L0, cada CSV gerado continua representando

os dados de uma única fonte para uma referência temporal específica. A fragmentação da L0 permite a tolerância a falhas na extração do texto antes de avançar para as validações estruturais.

- **Camada *Silver* (L1):** A transição para a camada L1 engloba a imposição de esquemas e a validação matemática. A granularidade do arquivo sofre sua primeira alteração arquitetural: o arquivo gerado na camada L1 passa a representar a união consolidada de todos os arquivos históricos extraídos da L0 Revisada. Contudo, essa consolidação ainda ocorre de maneira particionada por fonte de informação. Isso garante que qualquer falha de validação ou processamento em uma das origens possa ser identificada e isolada sem interferir na disponibilidade dos dados da outra fonte.

- **Camada *Gold* (L2):** É na penúltima camada de processamento que ocorre a consolidação global do *pipeline*. Os dados validados das diferentes origens (Fonte X e Fonte Y) convergem para formar um único documento mestre de ***data warehouse* (armazém central de dados)**. A granularidade torna-se abrangente e multiorigem.

Figura 2 – Fluxo de Processamento de Dados entre as Camadas



Fonte: Elaborado pelo autor.

5 A Implementação do *Pipeline*

Este capítulo detalha a implementação técnica da solução arquitetada, traduzindo os conceitos lógicos e os fluxos de informação apresentados anteriormente em rotinas de código. O foco desta seção é expor o ambiente de desenvolvimento, as diretrizes tecnológicas e a estruturação algorítmica utilizada para transitar, validar e consolidar os dados através da Arquitetura *Medallion*.

5.1 Ambiente de Desenvolvimento e Tecnologias

Todo o desenvolvimento do *pipeline* foi realizado utilizando a linguagem de programação Python. A definição do escopo tecnológico obedeceu a uma diretriz direta da equipe de desenvolvedores da Veeries, que estabeleceu o uso de bibliotecas específicas como requisito para garantir a integração e a padronização com o ecossistema de dados já existente na organização.

Para estruturar as transformações, o projeto foi construído sobre dois pilares de código aberto:

- **Pandas:** Biblioteca primária de manipulação e análise de dados em Python. Foi utilizada em todas as camadas do projeto para operações de união, cálculos de subtotais e limpeza de *strings*.
- **Pandera:** Biblioteca voltada para a validação estatística e estrutural de *DataFrames* (estruturas tabulares de dados). Atendendo à exigência da empresa por qualidade e governança, o Pandera foi a ferramenta empregada para materializar os "Contratos de Dados", garantindo a imposição de esquemas (*schema enforcement*) durante a execução do código.

O código-fonte foi versionado utilizando a plataforma *GitHub* e estruturado sob o paradigma de Programação Orientada a Objetos (POO), garantindo a modularidade e o reaproveitamento de rotinas entre diferentes origens de dados.

5.2 Orquestração e Ingestão (Camada *Bronze*)

O fluxo de execução do projeto é coordenado por *scripts* orquestradores (`main_source.py`), responsáveis por acionar os módulos sequencialmente e interromper o processo imediatamente caso ocorra uma falha crítica em qualquer camada.

A primeira etapa do *pipeline* é operada pelo módulo `l0_process.py`. Este componente atua como o motor de ingestão da Camada *Bronze* (L0). O seu papel principal é varrer os diretórios de entrada (`01_RAW`), identificar novos relatórios em formato nativo e acionar os algoritmos de extração de texto (*parsers*).

Optou-se por abstrair a complexidade visual dos relatórios originais logo no momento da ingestão. O *parser* localiza os dados tabulares e os metadados primários (como a data de referência do relatório) e o módulo L0 salva essas informações no formato .CSV, mantendo a integridade exata do que foi lido (L0 Original). O armazenamento nesta camada utiliza um particionamento lógico em arquivos compactados organizados por ano e mês, otimizando o espaço em nuvem e facilitando a auditoria temporal.

5.3 Validação e Padronização (Camada *Silver*)

A transição para a Camada *Silver* (L1) é o ponto crítico para a governança do *pipeline*, sendo executada pelo módulo `l1_process.py`. É nesta etapa que os dados brutos deixam de ser tratados apenas como "texto"e passam a ser validados como regras de negócio.

A implementação dessa validação baseia-se fortemente no arquivo `schemas.py`, onde classes de validação definem os contratos estruturais. A Figura 3 apresenta um trecho do código desenvolvido demonstrando a imposição de tipagem da camada L1.

Conforme ilustrado, o sistema força que todas as instâncias possuam campos tipados corretamente (como datas de referência obrigatórias e colunas de identificação de origem). Caso a fonte externa altere a estrutura do seu relatório, o processo de validação acusará uma violação de contrato e interromperá a execução, bloqueando a propagação do erro.

Além da tipagem, o módulo L1 realiza o mapeamento semântico. Através da leitura de dicionários de conversão, os nomes de categorias e variáveis que chegam despadronizados são normalizados para as nomenclaturas internas da empresa, centralizando o controle. A Figura 4 demonstra a estruturação de uma das lógicas de validação aplicadas no *pipeline*.

5.4 Transformação Analítica e Apresentação (Camadas *Gold* e *View*)

O refino final dos dados é dividido entre os módulos `l2_process.py` (Camada *Gold*) e `view_monthly_process.py` (Camada de Apresentação).

Na camada L2, a rotina executa a principal alteração estrutural do dado: a operação de *Melt* (despivoteamento). Essa técnica transforma variáveis dispostas horizontalmente (por exemplo, múltiplas colunas representando diferentes segmentações regionais ou catego-

Figura 3 – Trecho de Código - Implementação de Contrato de Dados com Pandera

```python
class L1Schema(pa.SchemaModel):
    dt_timestamp: Series[DateTime] = pa.Field(coerce=True, nullable=False)
    dt_report: Series[DateTime] = pa.Field(coerce=True, nullable=False)
    dt_reference: Series[DateTime] = pa.Field(coerce=True, nullable=False)
    dt_report_assistant: Series[DateTime] = pa.Field(coerce=True, nullable=False)
    dt_periodo: Series[DateTime] = pa.Field(coerce=True, nullable=False)
    dt_circular: Series[DateTime] = pa.Field(coerce=True, nullable=True)
    dt_path: Series[DateTime] = pa.Field(coerce=True, nullable=False)
    class_report: Series[String] = pa.Field(nullable=False)
    nm_path: Series[String] = pa.Field(nullable=False)
    source: Series[String] = pa.Field(
        nullable=False, isin=["Fonte X", "Fonte Y"]
    )

    # Business metadata
    nm_product: Series[String] = pa.Field(nullable=False)
    nm_group: Series[String] = pa.Field(nullable=False)
    sg_product: Series[String] = pa.Field(nullable=True)
    nm_product_secex: Series[String] = pa.Field(nullable=True)
    id_sh4: Series[String] = pa.Field(nullable=True)

    # Value columns - Nullable Integers (pd.Int64Dtype) to handle missing data
    total_port: Series[float] = pa.Field(nullable=True, coerce=True)
    year_over_year_change: Optional[Series[Float]] = pa.Field(coerce=True, nullable=True)
```

Fonte: Elaborado pelo autor.

rias) em linhas. Esse formato longo (*long format*) é o padrão exigido por ferramentas modernas de visualização. É também nesta camada que são processadas as agregações lógicas (subtotais sintéticos) e aplicadas as validações estritas de unicidade (*validate_uniqueness*) para prevenir a multiplicação indevida de instâncias geradas pela transformação.

Por fim, o módulo *View* abstrai toda a complexidade da engenharia para o usuário final. Ele atua removendo os metadados de auditoria técnica e executa o filtro de versão cronológica (*date versioning*). Caso as fontes emitam relatórios redundantes ou retificados no mesmo período, o código utiliza a data de extração para deduplicar a base, garantindo que a versão entregue aos analistas reflita estritamente o cenário consolidado mais atual e fidedigno do mercado.

5.5 Monitoramento e Rastreabilidade

Conforme ilustrado anteriormente pelas ramificações de interrupção na Figura 2, para garantir a confiabilidade da operação contínua do *pipeline* em ambiente de produção, o sistema não permite o avanço de dados corrompidos. Para auditar esses bloqueios, foi implementada uma arquitetura de monitoramento baseada em *logs* (registros detalhados

de execução).

Através da parametrização no módulo de configuração (`config.py`), o sistema inicializa um *logger* global padronizado, emitindo alertas simultâneos para o terminal de controle e para um arquivo físico de auditoria armazenado em nuvem junto aos dados brutos.

Durante a orquestração executada pelos *scripts* principais (`main_source.py`), as rotinas de processamento adotam a estratégia de propagação de erros em cascata (*cascading error propagation*). Isso significa que a execução das camadas é estritamente interdependente. Se o módulo da camada *Silver* identificar uma quebra no contrato de dados (uma coluna nova não mapeada enviada pela fonte, por exemplo), o erro estrutural é registrado no *log* e o *pipeline* é interrompido imediatamente.

A adoção dessa abordagem *fail-fast* (falhar rapidamente) é a etapa final de segurança arquitetural do projeto. Ela impede que um dado corrompido, não mapeado ou parcialmente validado avance e contamine as camadas *Gold* e de Apresentação. Dessa forma, assegura-se que a equipe de analistas de mercado consuma única e exclusivamente informações validadas, auditáveis e fidedignas, cumprindo o objetivo primário de qualidade de dados proposto no estágio.

Figura 4 – Trecho de Código - Rotina Lógica de Validação e Reconciliação

```python
def validate_horizontal_consistency(
    df: pd.DataFrame,
    cols_to_sum: List[str],
    col_target: str,
    col_filter: str,
    instances_to_exclude: List[str],
    tolerance: float,
    logger: logging.Logger
) -> Optional[pd.DataFrame]:
    """
    Realiza o cruzamento da soma de colunas específicas contra uma coluna de total/alvo.
    Args:
        df: DataFrame de entrada.
        cols_to_sum: Colunas que devem ser somadas para atingir o valor alvo.
        col_target: O total esperado.
        col_filter: Coluna usada para excluir linhas específicas (ex: totais já existentes).
        instances_to_exclude: Valores a serem ignorados na validação.
        tolerance: Erro absoluto permitido.
        logger: Instância do logger.
    Returns:
        None se for bem-sucedido.
    Raises:
        ValueError: Se as discrepâncias excederem a tolerância definida.
    """
    logger.debug(f"Verificando consistência horizontal para: {col_target}")

    if col_filter in df.columns:
        exclude_norm = [str(i).lower().strip() for i in instances_to_exclude]
        mask_valid = ~df[col_filter].astype(str).str.lower().str.strip().isin(exclude_norm)
        df_to_validate = df[mask_valid].copy()
    else:
        df_to_validate = df.copy()

    available_cols = [c for c in cols_to_sum if c in df_to_validate.columns]

    if not available_cols or col_target not in df_to_validate.columns:
        logger.warning(f"Validação ignorada: Coluna '{col_target}' ou colunas de origem ausentes.")
        return None

    # Conversão para numérico e tratamento de NaNs como zero
    temp_sum = df_to_validate[available_cols].apply(pd.to_numeric, errors='coerce').fillna(0)
    calc_sum = temp_sum.sum(axis=1)
    target_values = pd.to_numeric(df_to_validate[col_target], errors='coerce').fillna(0)

    diff = np.abs(calc_sum - target_values)
    inconsistent = diff > tolerance

    if inconsistent.any():
        num_errors = inconsistent.sum()
        df_errors = df_to_validate[inconsistent].copy()
        df_errors['calculated_sum'] = calc_sum[inconsistent]
        df_errors['target_value'] = target_values[inconsistent]

        msg = (f"Discrepância horizontal em '{col_target}'. "
               f"{num_errors} linhas falharam com tolerância de {tolerance}.")
        logger.error(msg)
        raise ValueError(msg)

    logger.debug(f"Checagem de consistência horizontal para '{col_target}': OK.")
    return None
```

Fonte: Elaborado pelo autor.

6 Capítulo 6

Análise dos resultados: análise crítica e profunda sobre o que fizeram, os resultados atingidos, os prós e contras, impactos da sua utilização na empresa, como "provam"que isso resolveu (ou melhorou) o problema inicialmente identificado, os indicadores usados para avaliação (e o porquê deles e não outros), os ganhos quanti ou qualitativos obtidos, etc..

Onde se apresentam e são discutidos os resultados (simulados, piloto ou reais) obtidos no decorrer da pesquisa/desenvolvimento. Nesta discussão é importante dar meios para que o leitor entenda os resultados obtidos e como foram obtidos. Deve-se fazer uma análise dos resultados: análise crítica profunda sobre o que foi feito, os resultados atingidos, os prós e contras, impactos, como isso resolveu o problema, os ganhos obtidos, etc.

7 Capítulo 7

Conclusões gerais: "resumão"do que foi feito e dos resultados globais, limitações do que foi desenvolvido e pressupostos assumidos, e sugestão para trabalhos futuros de continuação.

Síntese pessoal, objetiva, sucinta e interpretada dos resultados do trabalho. Grosso modo, deve-se apresentar um resumo do que foi feito, dos resultados globais (frente aos objetivos inicialmente traçados). Exemplos:

- o método deu certo? funcionou? deu o resultado esperado? Foi melhor que o método anterior?
- impactos organizacionais, tecnológicos, financeiros, éticos, ecológicos, etc., tidos (ou potencialmente a ter) com a introdução do que foi proposto.

De forma complementar, se pertinente, sugestões para trabalhos futuros de continuação.

Referências

ARMBRUST, M. et al. Delta lake: High-performance acid table storage over cloud object stores. *Proc. VLDB Endowment*, v. 13, n. 12, p. 3411–3424, 2020. Citado 2 vezes nas páginas 14 e 29.

Databricks. *What is a Medallion Architecture?* 2023. <https://www.databricks.com/glossary/medallion-architecture>. Acesso em: 19 mar. 2026. Citado 2 vezes nas páginas 12 e 13.

GORELIK, A. *The Enterprise Big Data Lake: Delivering the Promise of Big Data and Data Science*. [S.l.]: O'Reilly Media, 2019. Citado na página 13.

GUJJALA, P. K. R. Optimizing etl pipelines with delta lake and medallion architecture: A scalable approach for large-scale data. *International Journal for Multidisciplinary Research (IJFMR)*, 2023. Citado 5 vezes nas páginas 8, 12, 13, 28 e 29.

KIMBALL, R.; ROSS, M. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3. ed. [S.l.]: John Wiley & Sons, 2013. Citado na página 13.

KLEPPMANN, M. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. [S.l.]: O'Reilly Media, 2017. Citado na página 14.

LIAO, S. W.; ZHANG, X. Y.; LIN, F. Y. Schema evolution in big data systems: A review. *IEEE Access*, v. 8, p. 17456–17467, 2020. Citado 2 vezes nas páginas 14 e 29.

POLISETTY, S. M.; PENDYALA, S. K.; PRABHAKARAN, S. P. Enhancing data quality in streaming data pipelines using delta live tables. *International Journal of Research in Computer Applications and Information Technology (IJRCAIT)*, v. 6, n. 1, p. 87–96, 2023. Citado 3 vezes nas páginas 12, 14 e 29.

REIS, J.; HOUSLEY, M. *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*. [S.l.]: O'Reilly Media, 2022. Citado 4 vezes nas páginas 12, 13, 15 e 28.

APÊNDICE A – Glossário de termos técnicos

Este apêndice tem por objetivo apresentar as definições dos principais termos técnicos, conceitos e jargões da área de Engenharia de Dados utilizados ao longo do texto.

- **API (*Application Programming Interface*):** Conjunto de definições e protocolos que permite a comunicação e a integração entre diferentes sistemas ou aplicativos de *software*.
- **Arquitetura *Medallion*:** Padrão de projeto de organização de dados que divide os arquivos em camadas lógicas (comumente chamadas de *Bronze, Silver* e *Gold*), com o objetivo de melhorar progressivamente a estrutura e a qualidade das informações a cada etapa do processamento (GUJJALA, 2023).
- ***Business Intelligence* (BI):** Conjunto de teorias, metodologias e tecnologias (como o Power BI ou Tableau) que transformam dados brutos em informações significativas e úteis para a tomada de decisão estratégica do negócio.
- **Camada *Bronze* (L0):** Camada inicial de armazenamento na Arquitetura *Medallion* onde os dados brutos (*raw data*) são mantidos em seu estado original, exatamente como foram extraídos das fontes, sem aplicação de filtros ou transformações estruturais (GUJJALA, 2023).
- **Camada de Apresentação (*View*):** Camada lógica operando sobre a camada *Gold*, responsável por filtrar, deduplicar e expor os dados finais de maneira versátil para os analistas e ferramentas de visualização (REIS; HOUSLEY, 2022).
- **Camada *Gold* (L2):** Camada final onde os dados, já limpos e validados, são agregados e transformados com base em regras de negócio específicas, ficando prontos e otimizados para o consumo analítico e a geração de relatórios (GUJJALA, 2023).
- **Camada *Raw* (Crua):** Zona de pouso inicial (*landing zone*) de um *pipeline* de dados, responsável por receber e armazenar os arquivos externos exatamente em seu formato original, antes de qualquer processamento.
- **Camada *Silver* (L1):** Camada intermediária na qual os dados provenientes da camada *Bronze* passam por processos de limpeza, padronização, deduplicação e validação de qualidade contra contratos predefinidos (GUJJALA, 2023).

- **Contrato de Dados (*Data Contract*):** Acordo técnico e formal que define a estrutura, o formato, os padrões de qualidade e o comportamento esperado dos dados na transição entre sistemas ou camadas, garantindo a imposição de esquemas (*schema enforcement*) para a confiabilidade do sistema (ARMBRUST et al., 2020).

- **ETL (*Extract, Transform, Load*):** Processo tradicional de integração de dados que envolve a extração de informações de diversas fontes, sua transformação para um formato adequado e, por fim, o carregamento em um repositório centralizado, prática otimizada pelas modernas arquiteturas de *pipelines* (GUJJALA, 2023).

- **Granularidade:** Nível de detalhe, divisão ou resumo dos dados em uma estrutura. Ao longo de um *pipeline*, a granularidade pode transitar de estritamente unitária (um relatório por arquivo) para abrangente e consolidada (múltiplas fontes em um único repositório).

- **Ingestão de Dados:** Processo de coleta e importação de dados de diversas origens (como APIs, arquivos em lote ou bancos de dados) para um ambiente de armazenamento ou processamento primário (como a Camada *Bronze*) (POLISETTY; PENDYALA; PRABHAKARAN, 2023).

- ***Parser*:** Algoritmo ou rotina desenvolvida para analisar uma estrutura de dados de entrada (como um arquivo de texto, CSV ou planilha) e extrair os parâmetros e informações relevantes de forma legível para o processamento subsequente.

- ***Pipeline* de Dados:** Conjunto automatizado de processos e fluxos que realiza a extração, o processamento, a validação e o carregamento de dados de um sistema de origem para um ambiente de destino, garantindo o fluxo contínuo e a qualidade das informações (POLISETTY; PENDYALA; PRABHAKARAN, 2023).

- **Qualidade de Dados (*Data Quality*):** Medida da condição dos dados baseada em fatores como precisão, completude, consistência e confiabilidade, assegurada através de processos de validação e limpeza implementados durante o fluxo de dados (POLISETTY; PENDYALA; PRABHAKARAN, 2023).

- **Reconciliação de Dados (Validação Horizontal e Vertical):** Técnicas de auditoria matemática utilizadas para garantir que a soma de partes de um registro corresponda de forma exata aos totais informados pela fonte original, mitigando erros de extração (POLISETTY; PENDYALA; PRABHAKARAN, 2023).

- ***Schema* (Esquema):** Estrutura lógica que atua como a ferramenta de validação de um contrato de dados, especificando os nomes das colunas, os tipos de dados exigidos e as restrições de obrigatoriedade, sendo essencial para lidar com a evolução estrutural em sistemas de *Big Data* (LIAO; ZHANG; LIN, 2020).

- ***Webhook*:** Mecanismo de comunicação entre sistemas que permite que uma aplicação (como um *bot* do Telegram) envie informações ou comandos em tempo real para outra (como o GitHub) assim que um evento específico ocorre.